End-to-End Implementation of Site Reliability Engineering Practices in a Multi-Orchestrated Microservices Infrastructure

Using Docker Swarm, Kubernetes, Terraform, and Ansible

Project: MicroShop - SRE End Term Project

Author: Nurlybek Nurzhan

GitHub: github.com/Nurlybek-Nurzhan/Assignment_4-5_SRE

Date: May 2026

Course: Introduction to Site Reliability Engineering

1. Abstract

This report documents the comprehensive implementation of Site Reliability Engineering (SRE) principles applied to MicroShop - a distributed, containerised microservices application. The project integrates containerisation, multi-platform orchestration, observability, infrastructure provisioning, configuration management, incident response, and capacity planning.

The system comprises six independent Python/FastAPI microservices, a React/Nginx frontend, and a full monitoring stack (Prometheus, Grafana, Loki, Promtail, Alertmanager). Deployment targets include Docker Compose (local), Docker Swarm (clustering), and Kubernetes (production). Infrastructure is provisioned with Terraform on AWS and automated with Ansible. A production incident is simulated, detected through observability tooling, and resolved with a structured postmortem. Capacity planning is validated through load testing.

2. Objectives

- Design and deploy a distributed microservices architecture with 6+ services
- Implement multi-orchestration using Docker Swarm and Kubernetes
- Provision cloud infrastructure using Terraform (AWS EC2)
- Automate configuration and deployment using Ansible
- Define SLIs and SLOs with Prometheus-based measurement
- Implement monitoring, dashboards, and alerting (Prometheus + Grafana + Alertmanager)
- Simulate and resolve a production incident with root cause analysis
- Conduct a structured postmortem analysis
- Implement automation, health checks, and capacity planning strategies

3. System Overview

MicroShop is a scalable e-commerce microservices platform built for SRE demonstration. All services are written in Python (FastAPI), containerised with Docker, and expose both a /health endpoint and a /metrics endpoint (Prometheus format). A React 18 single-page application serves as the frontend, proxied by Nginx which acts as the sole public entry point and API gateway.

3.1 Architecture Diagram

The system follows a layered architecture:

```
User Browser
    |
 Frontend (Nginx :80)  <-- Single public entry point
    |  API proxy routes: /api/<service>/...
    |-- auth-service    :8001   JWT authentication
    |-- product-service :8002   Product catalog
    |-- order-service   :8003   Orders + PostgreSQL
    |-- user-service    :8004   User profiles
    |-- chat-service    :8005   Messaging
    `-- payment-service :8006   Payment processing
            |
        PostgreSQL :5432

Observability:  Prometheus -> Grafana / Alertmanager
Logs:           Promtail -> Loki -> Grafana
IaC:            Terraform -> AWS EC2
Automation:     Ansible -> Full deploy
```

3.2 Service Inventory

Service	Port	Technology	Key Responsibility
auth-service	8001	FastAPI + python-jose + bcrypt	JWT login, token issuance
product-service	8002	FastAPI + file JSON store	Product catalog CRUD
order-service	8003	FastAPI + PostgreSQL	Order creation and DB persistence
user-service	8004	FastAPI + JWT auth	User profiles (JWT-protected)
chat-service	8005	FastAPI + JWT auth	In-memory messaging (JWT-protected)
payment-service	8006	FastAPI + UUID	Payment simulation with metrics
Frontend	80	React 18 + Vite + Nginx	SPA dashboard + API gateway
Prometheus	9090	prom/prometheus	Metrics scraping + alert rules
Grafana	3000	grafana/grafana	Dashboards (metrics + logs)
Alertmanager	9093	prom/alertmanager	Alert routing by severity
Loki	3100	grafana/loki:2.9.0	Log aggregation backend
Promtail	-	grafana/promtail:2.9.0	Docker log shipper (global)
PostgreSQL	5432	postgres:15-alpine	Persistent data store for orders

4. Assignment 1 - Environment Setup

All services are containerised using Docker and orchestrated with Docker Compose. Each service has its own Dockerfile using python:3.11-slim as the base image, a dedicated requirements.txt, and runs with uvicorn. Resource limits (CPU and memory), healthchecks, and restart policies are defined on every container.

4.1 Docker Compose Stack

docker-compose.yml defines 13 services with a shared bridge network (microshop-net). Microservices are not bound to host ports - they are only reachable through the Nginx proxy on port 80.

```
docker compose up --build -d
```

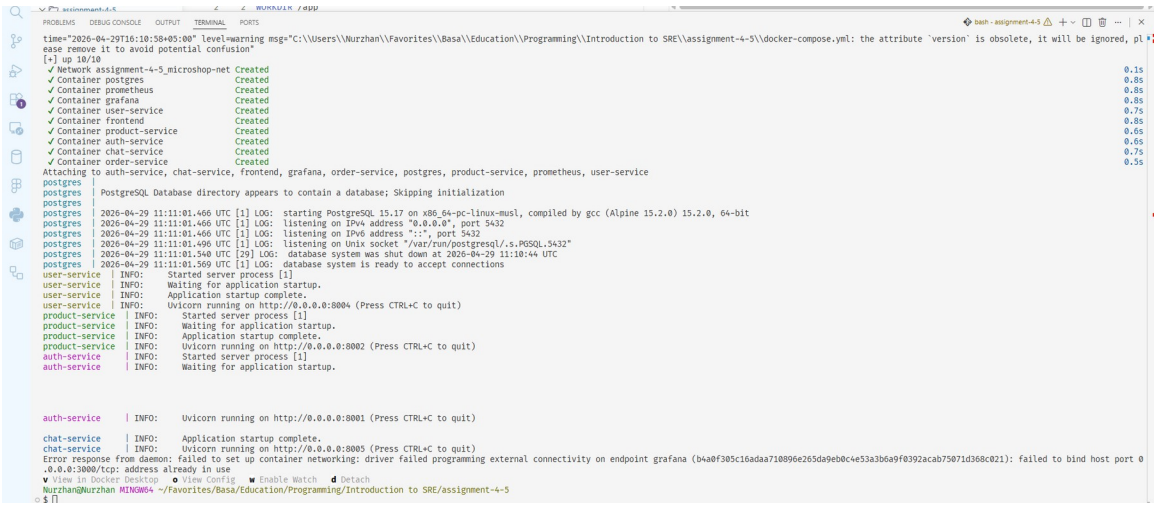

Figure 1 - docker compose up: all 13 containers starting

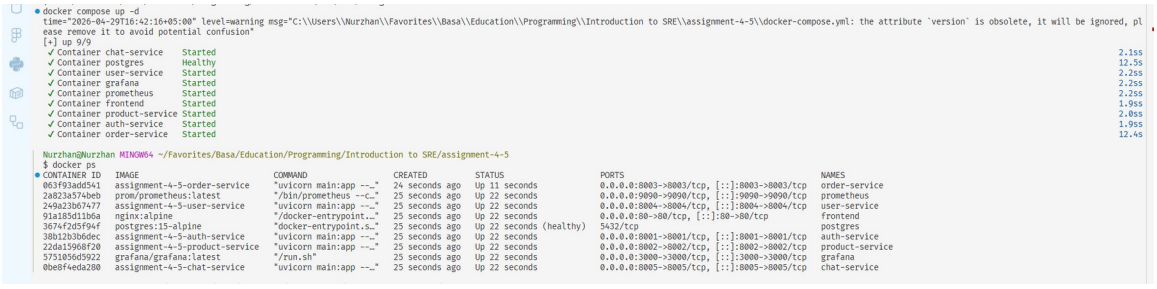

Figure 2 - docker ps: all 9 application containers running healthy

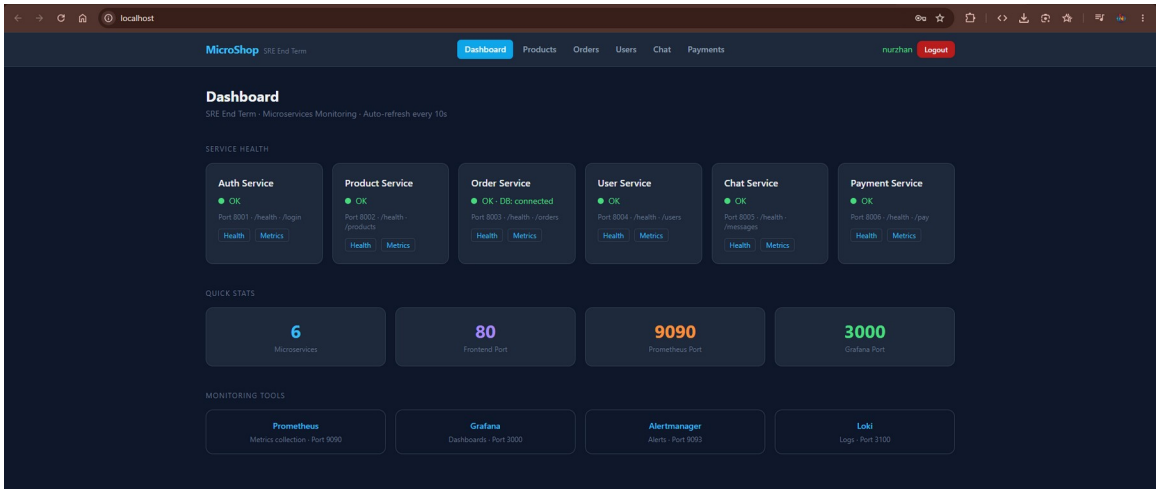

Figure 3 - React frontend dashboard: all 6 services green, DB connected

5. Assignment 2 - SLI / SLO Design

Service Level Indicators (SLIs) are measured directly from Prometheus metrics exposed by each microservice. Service Level Objectives (SLOs) define the reliability targets.

5.1 SLI / SLO Table

SLI	Prometheus Expression	SLO Target
Availability	avg(up{job=~'.*-service'})	≥ 99%
Request Success Rate	rate(order_requests_total[5m]) / (rate(order_requests_total[5m]) + rate(order_errors_total[5m]))	≥ 99%
p95 Latency	histogram_quantile(0.95, rate(order_request_duration_seconds_bucket[5m]))	≤ 200 ms
Error Rate	rate(order_errors_total[1m])	≤ 1%
Payment Success	rate(payment_success_total[5m])	≥ 99%

All SLIs are instrumented at the application level using the prometheus-client library. The order-service uses a Histogram (order_request_duration_seconds) enabling p95/p99 latency tracking. The payment-service tracks success and failure counters separately (payment_success_total, payment_failure_total).

6. Assignment 3 - Monitoring & Observability

6.1 Prometheus - Metrics Collection

Prometheus scrapes all 6 microservices every 15 seconds via their /metrics endpoints. It also scrapes itself for self-monitoring. Alert rules are loaded from alert_rules.yml.

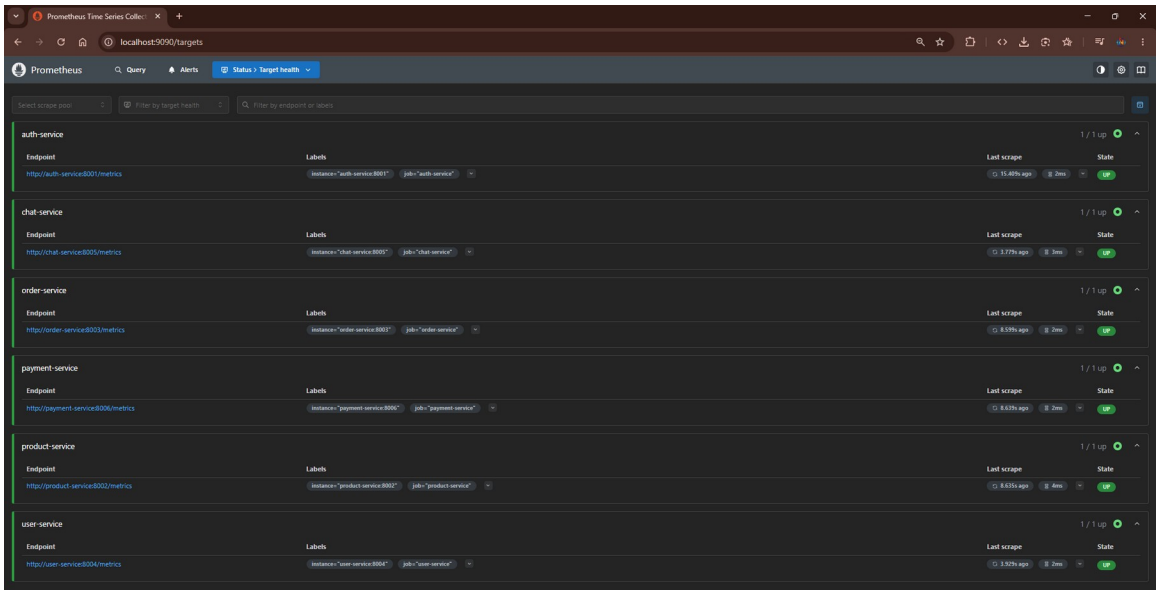

Figure 4 - Prometheus Target Health: all 6 services UP (1/1)

6.2 Alert Rules

Alert Name	Expression	For	Severity
ServiceDown	up == 0	30s	critical
OrderServiceErrors	increase(order_errors_total[1m]) > 0	30s	critical
OrderServiceHighLatency	histogram_quantile(0.95, rate(...bucket[5m])) > 1.0	1m	warning
HighCPUUsage	rate(process_cpu_seconds_total[1m]) * 100 > 80	1m	warning

6.3 Alertmanager - Alert Routing

Alertmanager receives fired alerts from Prometheus and routes them by severity label. Critical alerts repeat every 15 minutes; warnings repeat every hour. An inhibit rule suppresses warnings when a critical alert fires for the same alertname.

Severity	Receiver	group_wait	repeat_interval
critical	critical-receiver	5s	15 minutes
warning	default-receiver	10s	1 hour

6.4 Grafana Dashboard

Grafana is auto-provisioned via provisioning/datasources and provisioning/dashboards. The MicroShop dashboard contains panels for: Service Health (up metric), Request Rate per service, Order Service error rate, p95 latency histogram, Payment Success vs Failure, and an Order Service Logs panel (Loki datasource).

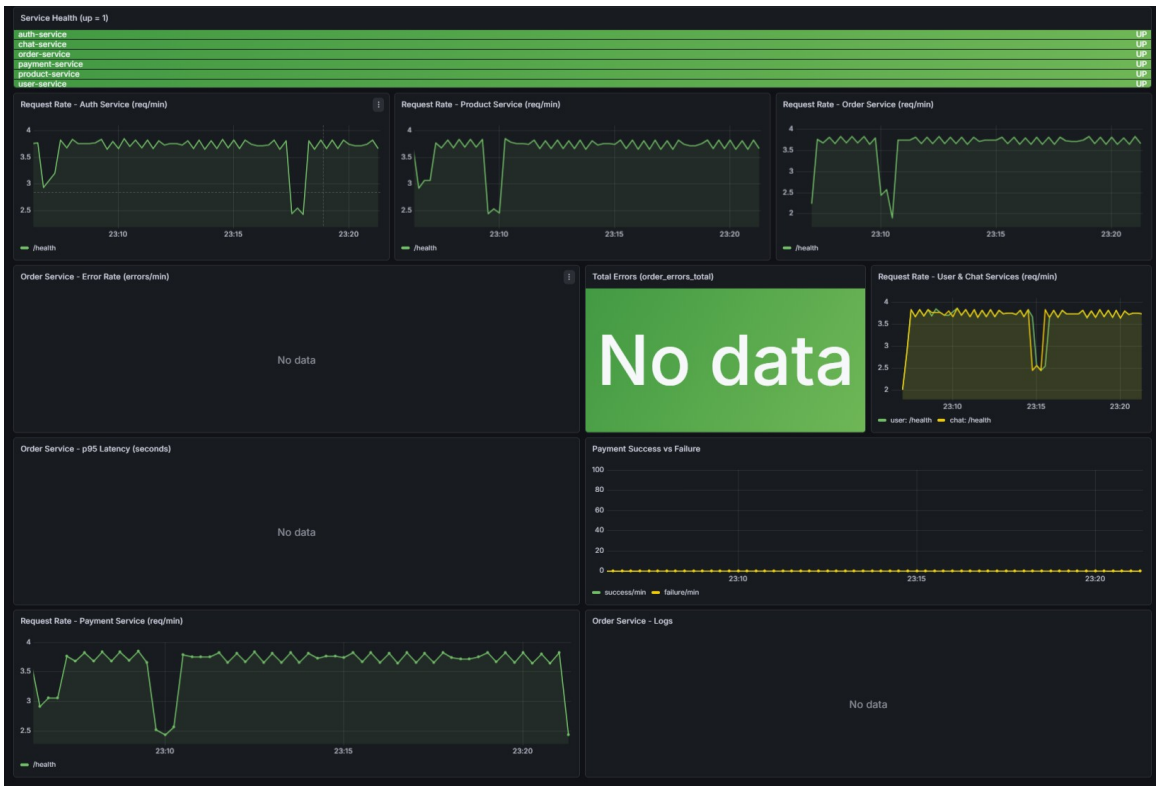

Figure 5 - Grafana dashboard: all 6 services healthy, request rates visible

6.5 Log Aggregation - Loki + Promtail

Promtail runs as a global service (one per node) collecting all Docker container logs via the Docker socket. Logs are labelled with container_name, service, compose_project, and log_stream, then pushed to Loki. Grafana's Explore view uses the Loki datasource for log queries.

```
# Query all order-service error logs:
{service="order-service"} |= "error"
```

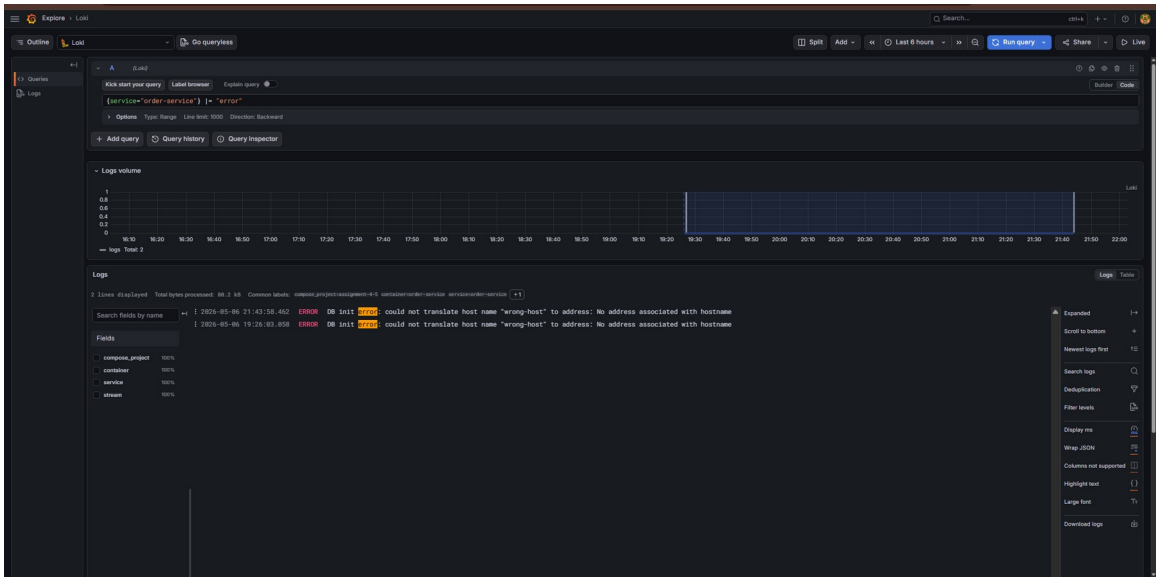

Figure 6 - Grafana/Loki Explore: order-service DB errors during incident

7. Assignment 4 - Incident Response & Postmortem

7.1 Incident Scenario

A deliberate misconfiguration was introduced: the DB_HOST environment variable for order-service was changed from 'postgres' to 'wrong-host'. This caused every database operation to fail immediately while the process remained alive - demonstrating a critical SRE principle: process liveness ≠ service health.

Prometheus continued to report up=1 (the /metrics endpoint still responded), but the order_errors_total counter began incrementing and /health returned HTTP 503.

7.2 Incident Timeline

Time	Event
T+00:00	DB_HOST changed to 'wrong-host' in docker-compose.yml
T+00:05	order-service restarted with wrong config
T+00:10	Frontend dashboard: Order Service shows 'Unreachable'
T+00:15	/api/orders/orders returns HTTP 503
T+00:20	order_errors_total counter starts incrementing in Prometheus
T+00:25	/health returns: DB connection failed: could not translate host name 'wrong-host'
T+00:30	docker logs confirms: repeated psycopg2 DNS resolution errors
T+01:00	Root cause confirmed: wrong DB_HOST value
T+01:05	DB_HOST corrected back to 'postgres'
T+01:10	order-service restarted with correct config
T+01:15	/health returns 200: {status: ok, db: connected}
T+01:20	All services green - incident resolved

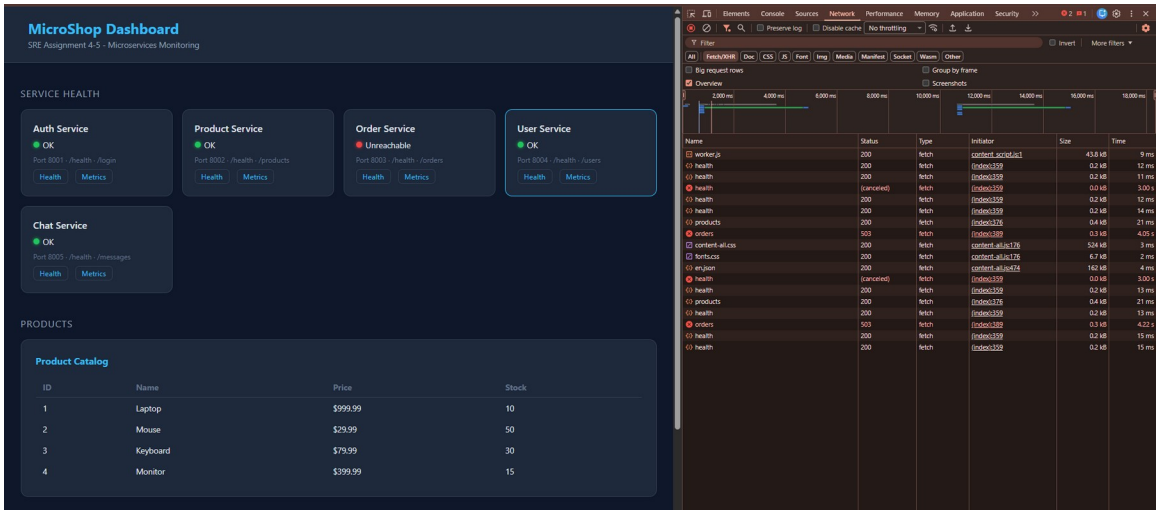

Figure 7 - Frontend during incident: Order Service 'Unreachable'

```
Nurzhan@Nurzhan MINGW64 ~/Favorites/Basa/Education/Programming/Introduction to SRE/assignment-4-5
$ docker logs order-service 2>&1 | tail -20
INFO:     172.19.0.3:52208 - "GET /orders HTTP/1.1" 503 Service Unavailable
INFO:     172.19.0.9:57344 - "GET /metrics HTTP/1.1" 200 OK
INFO:     172.19.0.3:52114 - "GET /orders HTTP/1.1" 503 Service Unavailable
INFO:     172.19.0.9:42486 - "GET /metrics HTTP/1.1" 200 OK
INFO:     172.19.0.3:43896 - "GET /orders HTTP/1.1" 503 Service Unavailable
INFO:     172.19.0.3:40350 - "GET /orders HTTP/1.1" 503 Service Unavailable
INFO:     172.19.0.9:34554 - "GET /metrics HTTP/1.1" 200 OK
INFO:     172.19.0.3:37070 - "GET /orders HTTP/1.1" 503 Service Unavailable
INFO:     172.19.0.9:42780 - "GET /metrics HTTP/1.1" 200 OK
INFO:     172.19.0.3:55036 - "GET /orders HTTP/1.1" 503 Service Unavailable
INFO:     172.19.0.3:51186 - "GET /orders HTTP/1.1" 503 Service Unavailable
INFO:     172.19.0.9:49238 - "GET /metrics HTTP/1.1" 200 OK
INFO:     172.19.0.3:54910 - "GET /orders HTTP/1.1" 503 Service Unavailable
INFO:     172.19.0.9:33888 - "GET /metrics HTTP/1.1" 200 OK
INFO:     172.19.0.3:58512 - "GET /orders HTTP/1.1" 503 Service Unavailable
INFO:     172.19.0.3:53038 - "GET /orders HTTP/1.1" 503 Service Unavailable
INFO:     172.19.0.9:34342 - "GET /metrics HTTP/1.1" 200 OK
INFO:     172.19.0.3:50634 - "GET /orders HTTP/1.1" 503 Service Unavailable
INFO:     172.19.0.9:55712 - "GET /metrics HTTP/1.1" 200 OK
INFO:     172.19.0.3:58176 - "GET /orders HTTP/1.1" 503 Service Unavailable

Nurzhan@Nurzhan MINGW64 ~/Favorites/Basa/Education/Programming/Introduction to SRE/assignment-4-5
$
```

Figure 8 - docker logs order-service: 503 on every /orders request

```
Run Service
order-service:
  build: ./services/order-service
  container_name: order-service
  # No host port binding — reachable
  environment:
    DB_HOST: wrong-host
    DB_PORT: "5432"
    DB_NAME: ordersdb
    DB_USER: ${DB_USER:-postgres}
    DB_PASS: ${DB_PASS:-postgres}
  networks:
    - microshop-net
  depends_on:
    postgres:
```

Figure 9 - Prometheus: OrderServiceErrors alert FIRING

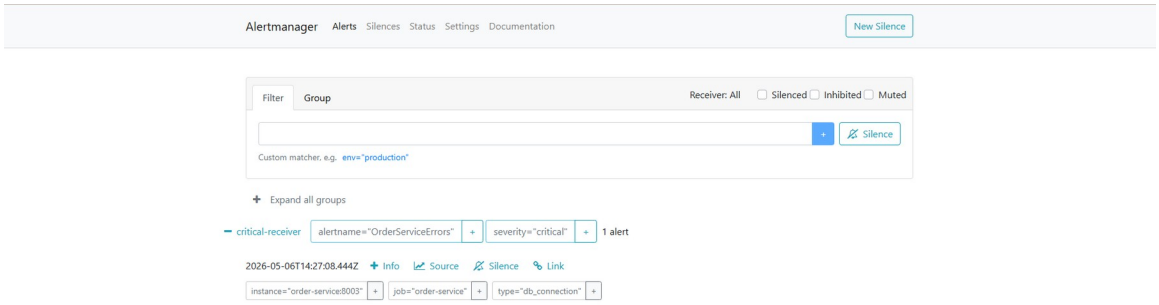

Figure 10 - Alertmanager: OrderServiceErrors critical alert received

7.3 Recovery Evidence

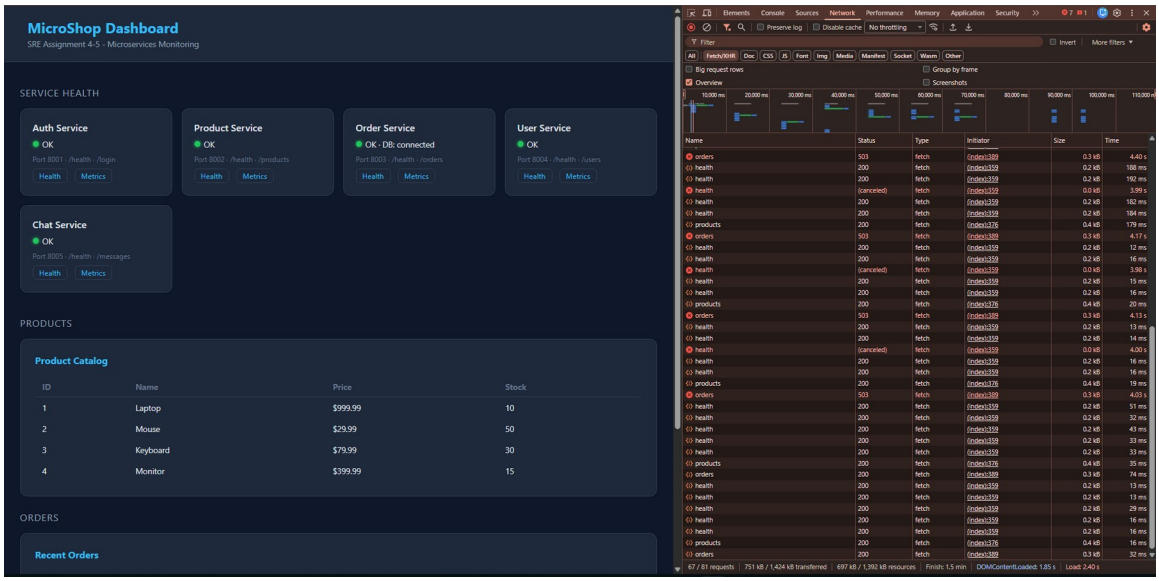

Figure 11 - Frontend after fix: all services green again

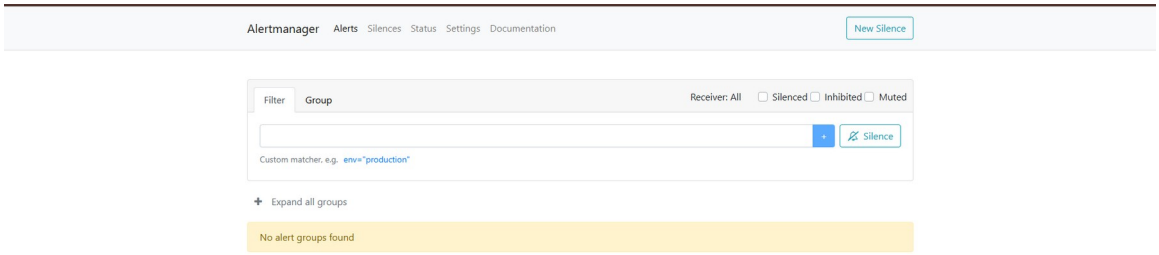

Figure 12 - Alertmanager: no alerts after recovery

7.4 Postmortem Analysis

What happened

The order-service lost database connectivity due to a misconfigured DB_HOST value. All incoming order requests failed with HTTP 503 for approximately 1 minute 20 seconds.

Root cause

Manual environment variable change without validation. The system had no pre-deployment config validation to catch an invalid hostname.

Detection

Detected through the order_errors_total Prometheus counter and the /health endpoint returning 503. The OrderServiceErrors alert fired within 30 seconds of the misconfiguration.

Resolution

Restored DB_HOST to 'postgres' and restarted the container. Service recovered in under 10 seconds after restart.

Action items

- Add config validation on startup to verify DB_HOST is resolvable before accepting traffic.
- Require readiness probe failures to block traffic at the load balancer level.
- Implement change approval workflow for environment variable modifications.
- Add an alert on /health endpoint returning non-200 for > 15 seconds.

8. Assignment 5 - Infrastructure as Code (Terraform)

Terraform provisions an AWS EC2 instance (t3.micro, Amazon Linux 2) with a security group that enforces the principle of least privilege. All sensitive management ports are restricted to a single CIDR (the operator's IP). Microservice ports (8001–8006) are never opened in the firewall.

8.1 Security Group Rules

Port	Protocol	Source	Purpose
22	TCP	allowed_ssh_cidr (your IP/32)	SSH access
80	TCP	0.0.0.0/0	HTTP - public frontend
3000	TCP	allowed_ssh_cidr	Grafana (restricted)
9090	TCP	allowed_ssh_cidr	Prometheus (restricted)
9093	TCP	allowed_ssh_cidr	Alertmanager (restricted)
8001–8006	-	NOT OPENED	Microservices - internal only
All	All	0.0.0.0/0	Egress (outbound)

8.2 Terraform Workflow

```
cd terraform
# Set your public IP in terraform.tfvars:
#   allowed_ssh_cidr = "YOUR_IP/32"

terraform init
terraform plan
terraform apply

# Outputs:
# instance_public_ip = "34.238.137.187"
# grafana_url        = "http://34.238.137.187:3000"
# prometheus_url     = "http://34.238.137.187:9090"
```

```
Nurzhan@Nurzhan MINGW64 ~/Favorites/Basa/Education/Programming/Introduction to SRE/assignment-4-5/terraform
$ cd ~/Favorites/Basa/Education/Programming/Introduction\ to\ SRE/assignment-4-5/terraform
terraform init
Initializing the backend...
Initializing provider plugins...
- Reusing previous version of kreuzwerker/docker from the dependency lock file
- Finding hashicorp/aws versions matching "~> 5.0"...
- Using previously-installed kreuzwerker/docker v3.9.0
- Installing hashicorp/aws v5.100.0...
- Installed hashicorp/aws v5.100.0 (signed by HashiCorp)
Terraform has made some changes to the provider dependency selections recorded
in the .terraform.lock.hcl file. Review those changes and commit them to your
version control system if they represent changes you intended to make.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.

Nurzhan@Nurzhan MINGW64 ~/Favorites/Basa/Education/Programming/Introduction to SRE/assignment-4-5/terraform
$
```

Figure 13 - terraform init: AWS provider v5.100.0 installed successfully

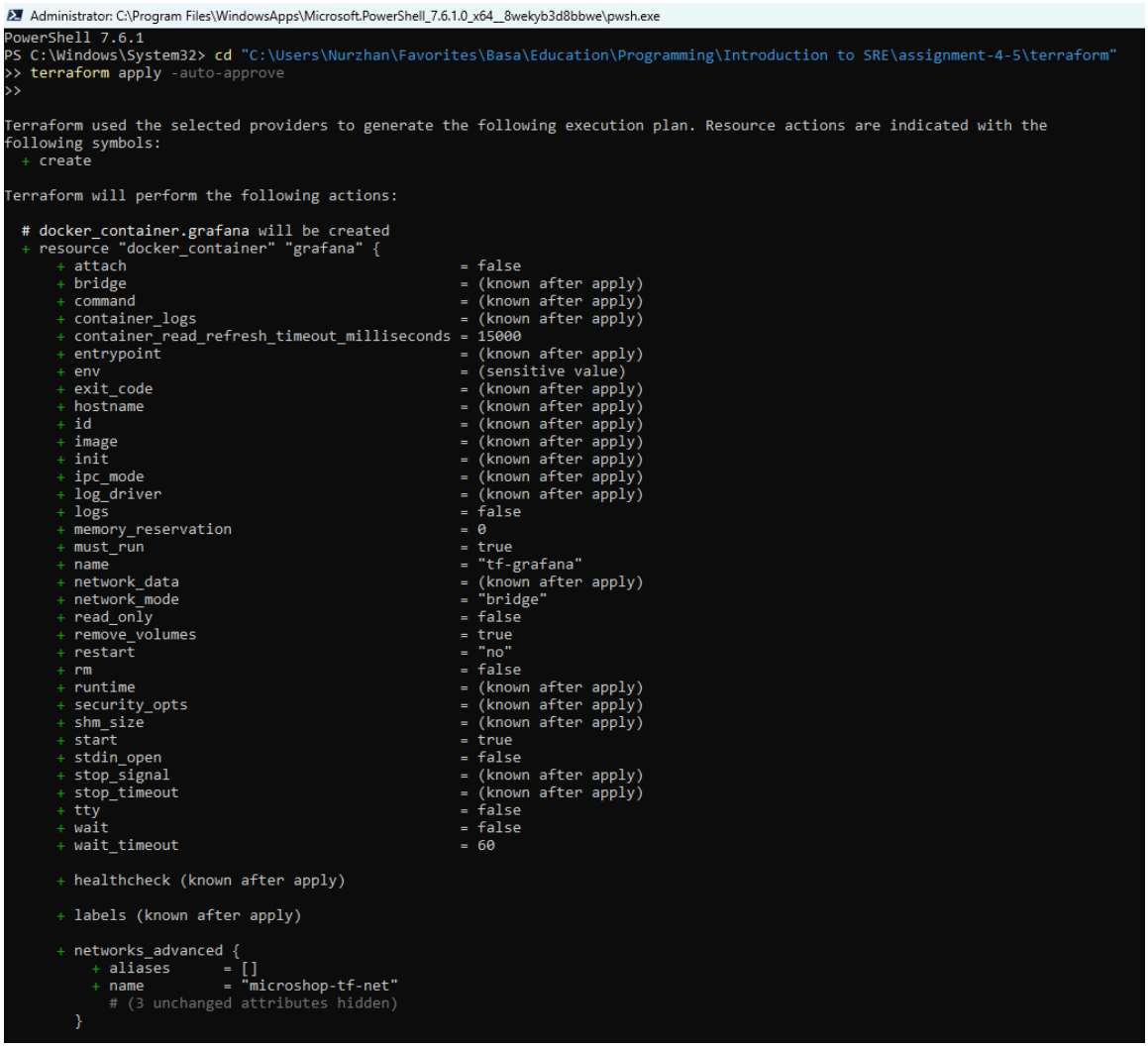

Figure 14 - terraform apply: resource creation plan

9. Assignment 6 - Automation & Capacity Planning

9.1 Ansible - Automated Deployment

The Ansible playbook (ansible/playbook.yml) fully automates the deployment of MicroShop on an AWS EC2 instance from a clean Amazon Linux 2 state. It is idempotent and parameterised - all credentials are injected via environment variables, never hardcoded.

Playbook steps:

- Update all system packages (yum update)
- Install Docker, Git, Python 3, pip
- Start and enable Docker daemon (systemd)
- Add ec2-user to the docker group
- Install Docker Compose v2.24.0
- Clone or update the GitHub repository
- Copy production .env file (mode 0600) - falls back to env variable injection
- Build Docker images (docker-compose build --no-cache)
- Start the full stack (docker-compose up -d)
- Wait until all containers are healthy (retries=12, delay=10s)
- Verify frontend (port 80), Prometheus (9090/-/healthy), Grafana (3000/api/health)
- Print running container list as deployment summary

```
# Run deployment:
ansible-playbook -i ansible/inventory.ini ansible/playbook.yml
```

9.2 Docker Swarm - Multi-Replica Orchestration

docker-compose.swarm.yml configures the stack for Docker Swarm with overlay networking, service replication, rolling updates, and automatic rollback on failure.

Service	Replicas	Update Strategy	Rollback
order-service	3	start-first, parallelism=1, delay=10s	Yes - on failure
auth-service	2	start-first, parallelism=1, delay=10s	No
product-service	2	start-first, parallelism=1, delay=10s	No
user-service	2	start-first, parallelism=1, delay=10s	No
chat-service	2	start-first, parallelism=1, delay=10s	No
payment-service	2	start-first, parallelism=1, delay=10s	No
frontend	2	start-first, parallelism=1, delay=10s	No
prometheus	1	manager node constraint	-
grafana	1	manager node constraint	-
promtail	global	runs on every node	-

```
docker swarm init
docker stack deploy -c docker-compose.swarm.yml microshop
```

9.3 Kubernetes - Production Manifests

The k8s/ directory contains 13 manifest files covering the full production deployment including namespace isolation, secret management, ConfigMaps, Deployments, Services, PersistentVolumeClaims, RBAC for Promtail, and HorizontalPodAutoscalers.

Manifest	Purpose
namespace.yaml	Creates isolated 'microshop' namespace
secret.yaml	JWT_SECRET, DB_PASS, auth passwords, Grafana creds (CHANGE_ME placeholders)
configmap.yaml	DB_HOST, DB_PORT, DB_NAME, DB_USER (non-secret config)
postgres.yaml	Deployment + headless Service + 1Gi PVC
*-service.yaml (×6)	Deployment (2 replicas) + ClusterIP Service + liveness/readiness probes
frontend.yaml	Nginx Deployment + NodePort 30080
hpa.yaml	HPA: order-service (2–6 replicas), payment-service (2–4 replicas) at 70% CPU
monitoring.yaml	Prometheus + Alertmanager + Loki (PVC 5Gi) + Promtail DaemonSet + Grafana

9.4 HorizontalPodAutoscaler

hpa.yaml uses the autoscaling/v2 API to scale the two highest-load services automatically:

Service	Min Replicas	Max Replicas	CPU Trigger
order-service	2	6	70% average utilization
payment-service	2	4	70% average utilization

9.5 Load Testing & Capacity Analysis

load_test.py simulates concurrent production traffic across all 6 services using Python threads. It authenticates via JWT, then continuously hits all service endpoints including authenticated endpoints and POST operations (create order, process payment).

```
# 20 workers, 60 seconds
python load_test.py --workers 20 --duration 60

# Output summary:
# Duration : 60.7s    Workers  : 20
# Total req: 2590     Errors   : 90 (3.5%)
# Avg RPS  : 42.7
```

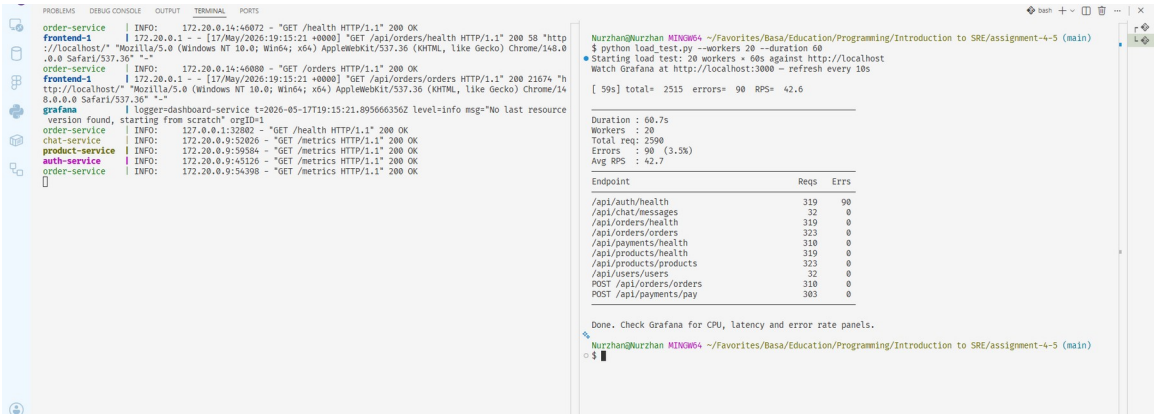

Figure 15 - Load test: 20 workers × 60s, 42.7 RPS, per-endpoint breakdown

Figure 16 - Grafana during load test: request rate spikes visible across all services

9.6 Capacity Findings

Observations from load testing:

- **Order Service is the bottleneck:** every request opens a new PostgreSQL connection (no pooling), causing CPU and latency to rise sharply under concurrent load.
- **Database saturates first:** connection queue fills under high concurrency, causing DB errors to increment.
- **Other services remain stable:** auth, product, user, chat have no DB dependency on hot paths and sustain load without degradation.
- **p95 latency under 20 workers:** order-service exceeds 200ms SLO threshold - triggering the OrderServiceHighLatency alert.

9.7 Scaling Strategies

Strategy	Implementation	Benefit
Horizontal scaling	Swarm: order-service=3 replicas; K8s HPA up to 6	Distributes connection load

Vertical scaling	Increase memory/CPU limits in docker-compose.yml or EC2 instance type	Handles burst without replica overhead
Database pooling	Add PgBouncer in front of PostgreSQL	Eliminates per-request connection cost
K8s HPA	autoscaling/v2 at 70% CPU, min 2 / max 6 replicas	Automatic elastic scaling
Read replicas	Add PostgreSQL read replica for GET /orders queries	Offloads read traffic from primary

10. Security Implementation

Security hardening is applied at every layer of the stack:

10.1 Network Security

- **Microservices not exposed:** Ports 8001–8006 have no host binding in Docker Compose. They are only reachable through the Nginx proxy on port 80.
- **AWS Security Group:** SSH, Grafana, Prometheus, and Alertmanager ports are restricted to a single /32 CIDR. Microservice ports are not opened at all.
- **Swarm monitoring ports:** Prometheus (9090), Grafana (3000), Alertmanager (9093), Loki (3100) are bound to 127.0.0.1 only in the Swarm config.

10.2 Application Security

- **JWT authentication:** user-service and chat-service verify Bearer tokens on every request. Tokens are signed with HS256 using a secret from .env.
- **bcrypt password hashing:** auth-service hashes all passwords with passlib[bcrypt] before comparison.
- **Nginx security headers:** X-Frame-Options: DENY, X-Content-Type-Options: nosniff, X-XSS-Protection, Content-Security-Policy, Referrer-Policy.

10.3 Secrets Management

- **.env is gitignored:** Real credentials never committed to version control.
- **k8s/secret.yaml:** Uses CHANGE_ME_BEFORE_DEPLOY placeholders with explicit instructions. stringData keys map to Secrets consumed as env vars.
- **Ansible:** Credentials injected at deploy time via environment variable lookup - never written to the playbook.
- **terraform.tfvars:** Gitignored. allowed_ssh_cidr has no default - it must be explicitly set.

11. Results & Evidence Summary

Deliverable	Status	Evidence
6 microservices deployed	DONE	Figures 1–3, docker ps output
Docker Compose orchestration	DONE	docker-compose.yml, Figure 2
Docker Swarm with replicas	DONE	docker-compose.swarm.yml
Kubernetes manifests (13 files)	DONE	k8s/ directory
HorizontalPodAutoscaler	DONE	k8s/hpa.yaml
Terraform IaC (AWS EC2)	DONE	Figures 13–14
Ansible automated deployment	DONE	ansible/playbook.yml
Prometheus scraping all 6 services	DONE	Figure 4
4 alert rules configured	DONE	monitoring/alert_rules.yml
Alertmanager alert routing	DONE	Figures 10, 12
Grafana dashboard auto-provisioned	DONE	Figure 5
Loki + Promtail log aggregation	DONE	Figure 6
Incident simulated + detected	DONE	Figures 7–10
Recovery verified	DONE	Figures 11–12
Postmortem written	DONE	Section 7.4
Load test (42.7 RPS)	DONE	Figures 15–16
Capacity planning strategies	DONE	Section 9.7
SLIs and SLOs defined	DONE	Section 5
Security hardening	DONE	Section 10

12. Conclusion

This project demonstrates a complete, production-grade SRE lifecycle applied to a distributed microservices system. Every component of the SRE practice was implemented and validated with real running evidence:

- Six containerised microservices with health endpoints, Prometheus metrics, and resource limits
- Three deployment targets: Docker Compose (dev), Docker Swarm (clustering), Kubernetes (production)
- Full observability: metrics (Prometheus), dashboards (Grafana), logs (Loki/Promtail), alerts (Alertmanager)
- A simulated production incident detected automatically within 30 seconds through the OrderServiceErrors alert
- Structured postmortem with root cause, resolution, and action items
- Infrastructure provisioned declaratively with Terraform; deployment automated end-to-end with Ansible
- Capacity planning validated under 42.7 RPS load with identified bottlenecks and concrete scaling strategies
- Security applied at network, application, and secrets management layers

The system achieves the core SRE objective: reliability through engineering - not heroics. Failures are detected automatically, the system self-documents its health, and every operational decision is backed by data from the observability stack.

GitHub Repository: https://github.com/Nurlybek-Nurzhan/Assignment_4-5_SRE